Experimental Report on the School of Computer and Artificial Intelligence at Zhengzhou University

Grade：2024 Major：SOFTWARE ENGINEERING Class：GROUP-1

Name: MD TASNIMUL HASAN Student Number：202480090104

Instructor：ALEX Score： 2026 year 10 month 12 date

Experimental Topic：Widget Components

1. Experimental Purpose

The main objectives of this experiment are:

1. To become proficient in using Android's core UI widget components, including ScrollView, ProgressBar, SeekBar, RatingBar, ImageView, ImageButton, Switch, GridView, and TextView.
2. To learn how to design a complex Android user interface by combining multiple widgets using LinearLayout and ScrollView.
3. To understand how to handle user interaction events through listeners in Kotlin.
4. To learn how to build a custom Adapter for a GridView in order to display dynamic data.
5. To develop practical skills in designing, coding, building, and running a complete Android application.

2. Experimental Content

In this experiment, an Android application named **"WidgetComponents"** was designed and implemented. The application demonstrates the use of the following widget components: **ScrollView** (for vertical scrolling), **TextView** (for both static and dynamic text), **ImageView** (for displaying an image), **ImageButton** (a clickable image), **ProgressBar** (for showing progress), **SeekBar** (a draggable value control), **RatingBar** (for 1–5 star rating), **Switch** (an ON/OFF toggle), and **GridView** (for displaying 9 items in a 3-column grid using a custom adapter).

3. Experimental Environment

The experiment was carried out on **Windows 11** using **Android Studio** as the IDE, with **Kotlin** as the programming language and **Gradle** as the build system. The application targets a minimum SDK of **API 21 (Android 5.0)** and was compiled with **API 37.2 (Android 17.0)**. Testing was performed on a **Pixel 6 emulator** (1080×2400, 420 dpi) running the **Google APIs x86_64 (API 37.2)** system image. The project was named **WidgetComponents**.

4. Procedure

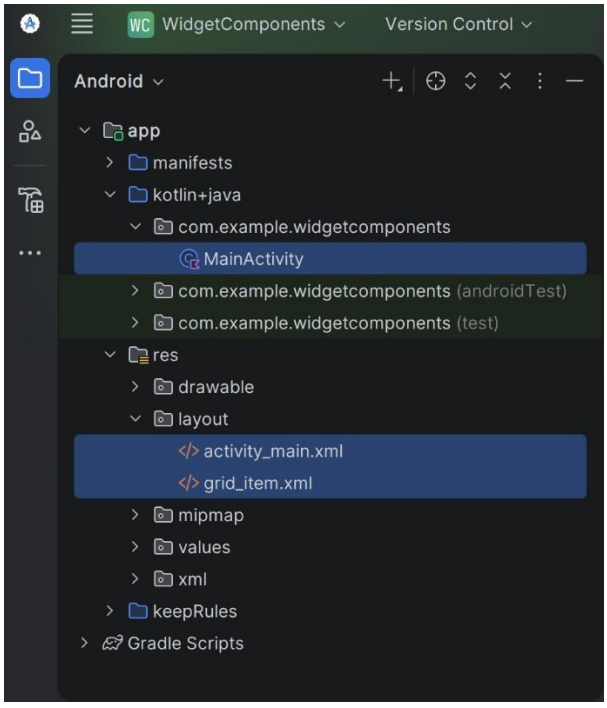
The experiment was carried out in the following systematic steps:

4.1 Creating the Project Structure

A new Android project was created in Android Studio using the Empty Views Activity template, with Kotlin as the programming language. The project contains three main files:

- MainActivity.kt — Kotlin source code
- activity_main.xml — Main UI layout
- grid_item.xml — GridView cell layout

Figure 1: Project Structure

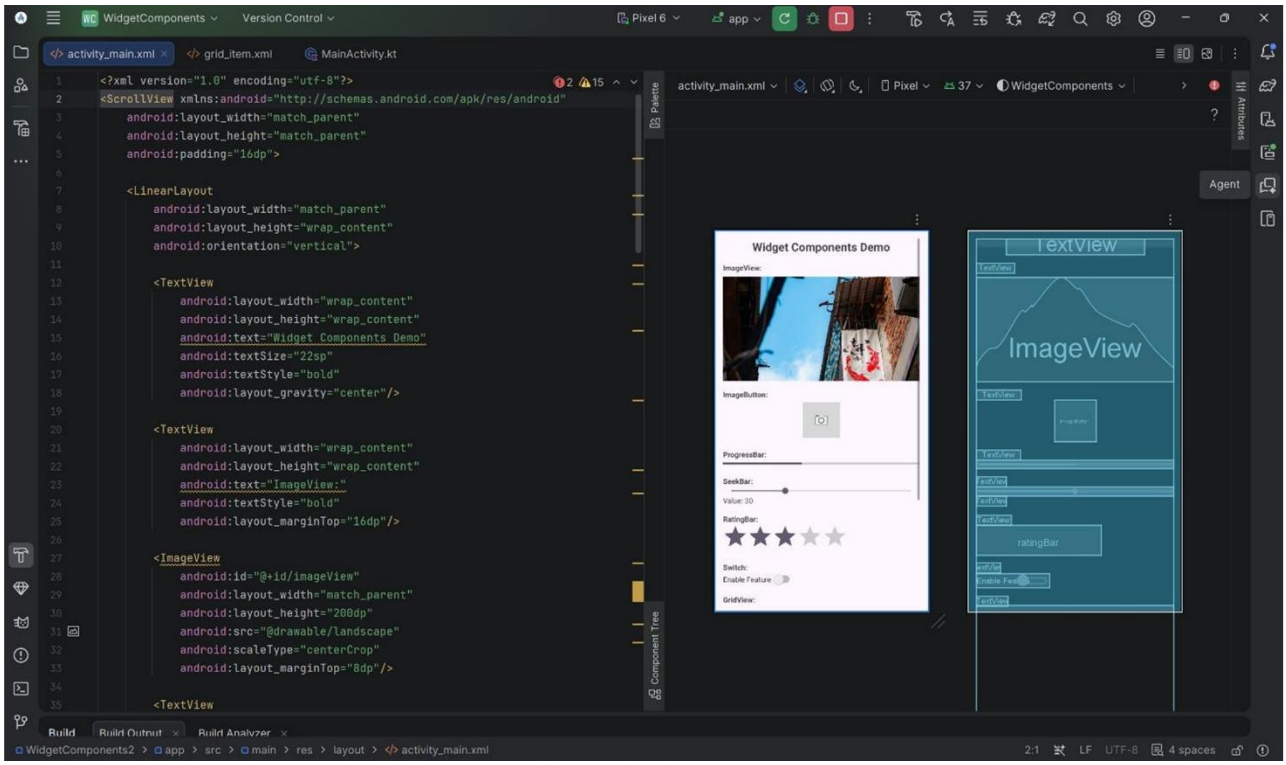


4.2 Designing the Main UI Layout (activity_main.xml)

The main layout uses a **ScrollView** as the root container, with a vertical **LinearLayout** nested inside, where all widgets are arranged vertically. The key components are:

- **ScrollView** — scrollable root container
- **LinearLayout (vertical)** — holds all child widgets
- **ImageView**, **ImageButton** — for displaying and interacting with images
- **ProgressBar (horizontal)**, **SeekBar** for showing and controlling progress values
- **TextView** (label + dynamic value) — for static labels and updating values
- **RatingBar**, **Switch** — for star rating and ON/OFF toggling
- **GridView (3 columns)** — for displaying items in a grid layout

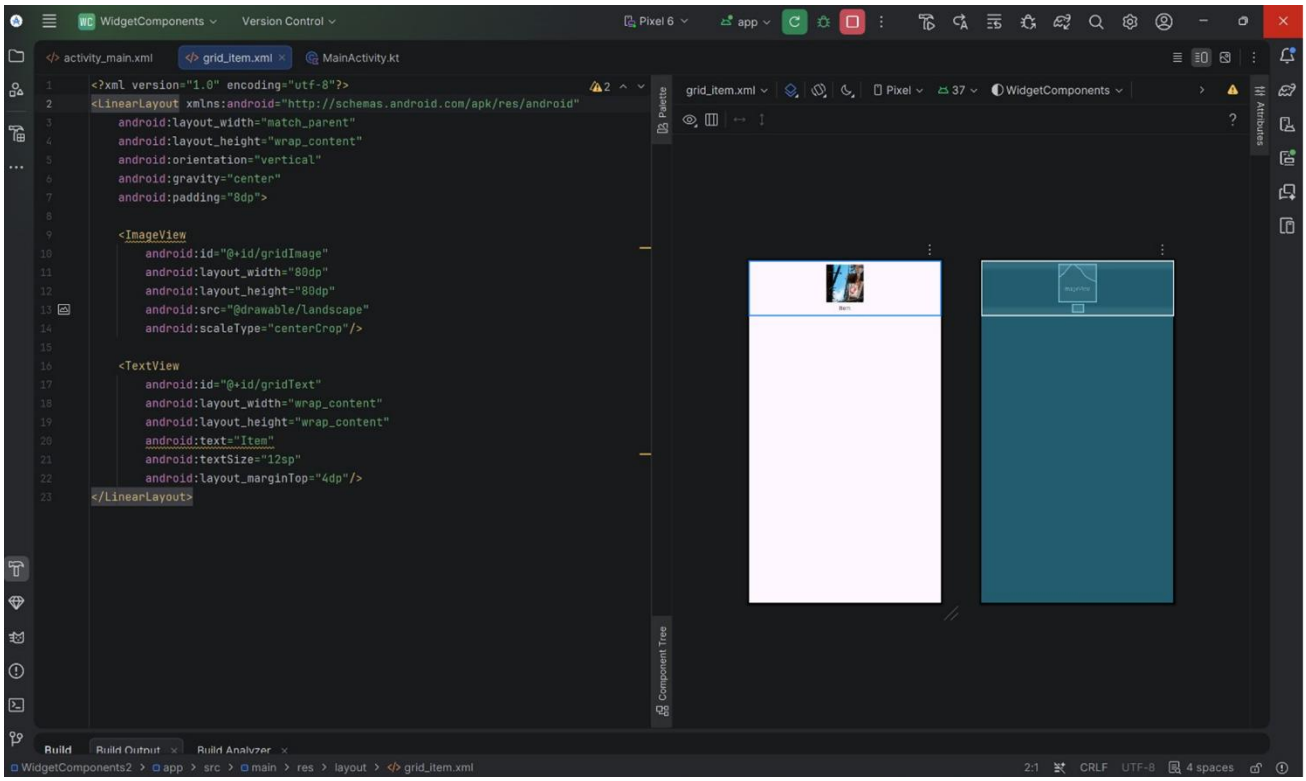
Figure 2: activity_main.xml Code



4.3 Designing the GridView Cell Layout (grid_item.xml)

A separate layout file was created for each GridView cell, containing a vertical LinearLayout with an ImageView and a TextView.

Figure 3: grid_item.xml Code



4.4 Writing the Kotlin Logic (MainActivity.kt)

The Kotlin code performs the following tasks:

- Links all widgets using findViewById()
- Sets up event listeners for each interactive widget
- Implements a custom GridAdapter class extending BaseAdapter
- Inflates each GridView cell using layoutInflater.inflate()

The code is presented below in parts:

Figure 4.1: Package, Imports, and Variable Declarations

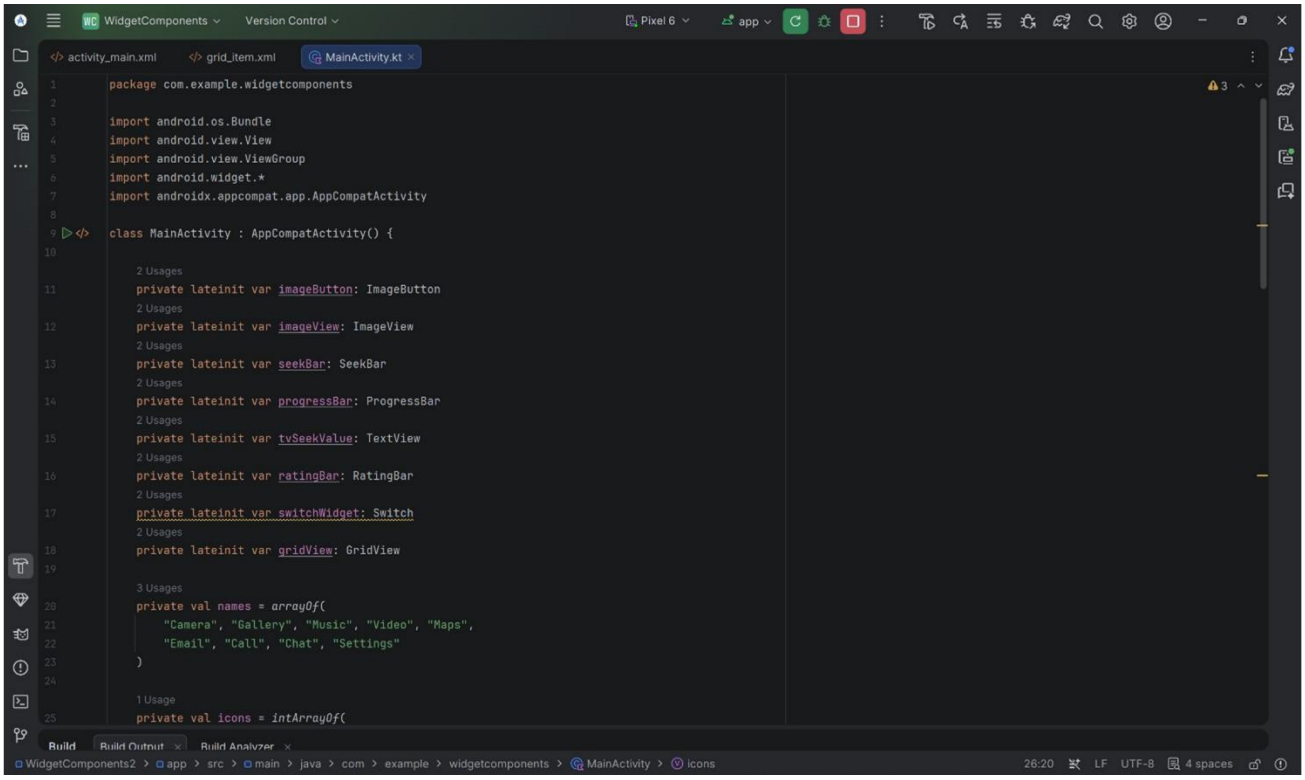


Figure 4.2: Icons Array and onCreate() Method

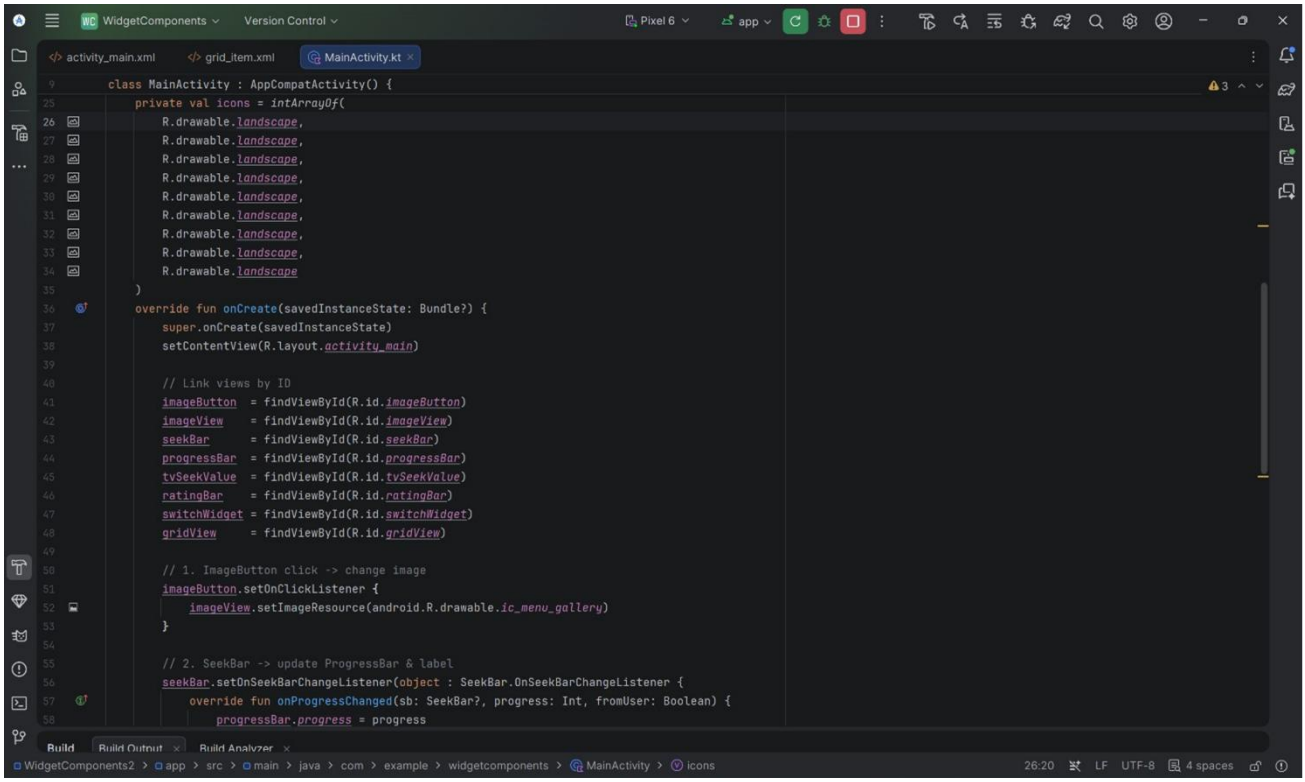
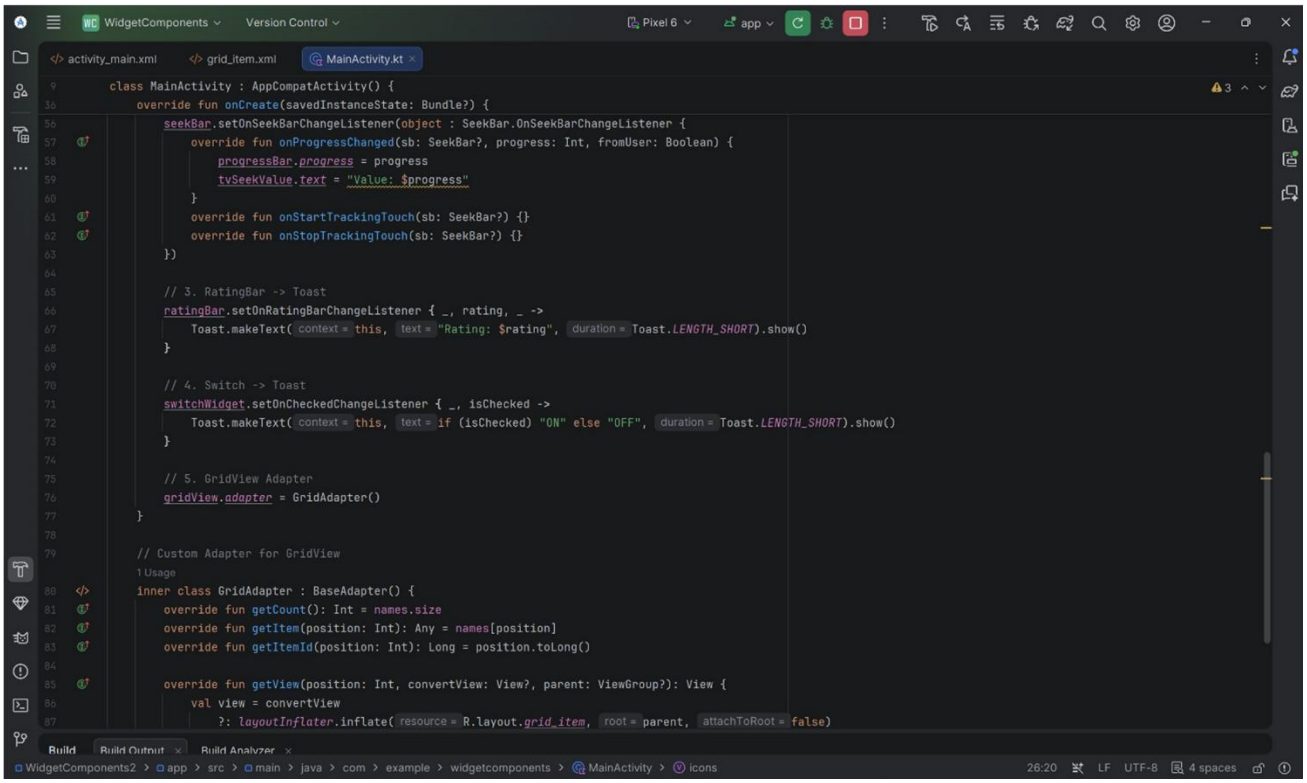


Figure 4.3: Event Listeners (SeekBar, RatingBar, Switch)



```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        seekBar.setOnSeekBarChangeListener(object : SeekBar.OnSeekBarChangeListener {
            override fun onProgressChanged(sb: SeekBar?, progress: Int, fromUser: Boolean) {
                progressBar.progress = progress
                tvSeekValue.text = "Value: $progress"
            }
            override fun onStartTrackingTouch(sb: SeekBar?) {}
            override fun onStopTrackingTouch(sb: SeekBar?) {}
        })

        // 3. RatingBar -> Toast
        ratingBar.setOnRatingBarChangeListener { _, rating, _ ->
            Toast.makeText( context = this, text = "Rating: $rating", duration = Toast.LENGTH_SHORT).show()
        }

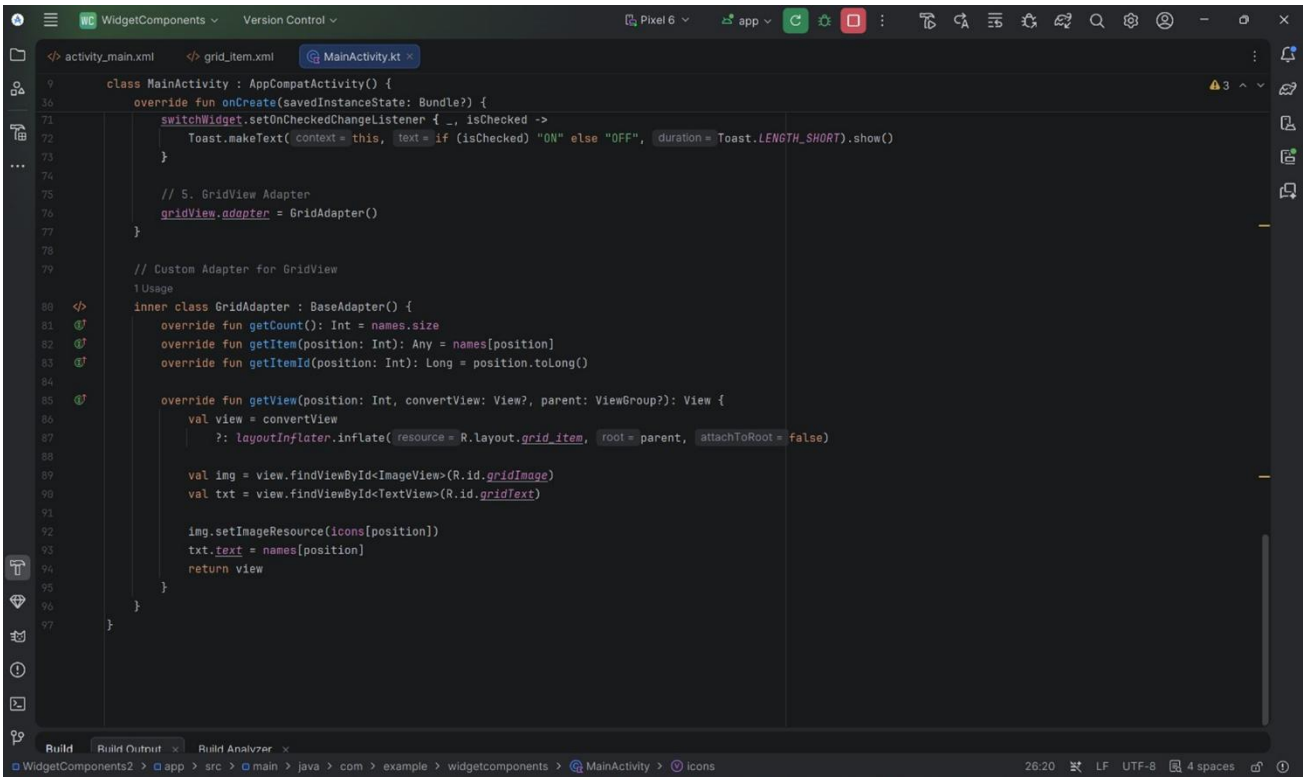
        // 4. Switch -> Toast
        switchWidget.setOnCheckedChangeListener { _, isChecked ->
            Toast.makeText( context = this, text = if (isChecked) "ON" else "OFF", duration = Toast.LENGTH_SHORT).show()
        }

        // 5. GridView Adapter
        gridView.adapter = GridAdapter()
    }

    // Custom Adapter for GridView
    1 Usage
    inner class GridAdapter : BaseAdapter() {
        override fun getCount(): Int = names.size
        override fun getItem(position: Int): Any = names[position]
        override fun getItemId(position: Int): Long = position.toLong()

        override fun getView(position: Int, convertView: View?, parent: ViewGroup?): View {
            val view = convertView
                ?: layoutInflater.inflate( resource = R.layout.grid_item, root = parent, attachToRoot = false)
```

Figure 4.4: Custom GridAdapter Class



```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        switchWidget.setOnCheckedChangeListener { _, isChecked ->
            Toast.makeText( context = this, text = if (isChecked) "ON" else "OFF", duration = Toast.LENGTH_SHORT).show()
        }

        // 5. GridView Adapter
        gridView.adapter = GridAdapter()
    }

    // Custom Adapter for GridView
    1 Usage
    inner class GridAdapter : BaseAdapter() {
        override fun getCount(): Int = names.size
        override fun getItem(position: Int): Any = names[position]
        override fun getItemId(position: Int): Long = position.toLong()

        override fun getView(position: Int, convertView: View?, parent: ViewGroup?): View {
            val view = convertView
                ?: layoutInflater.inflate( resource = R.layout.grid_item, root = parent, attachToRoot = false)

            val img = view.findViewById<ImageView>(R.id.gridImage)
            val txt = view.findViewById<TextView>(R.id.gridText)

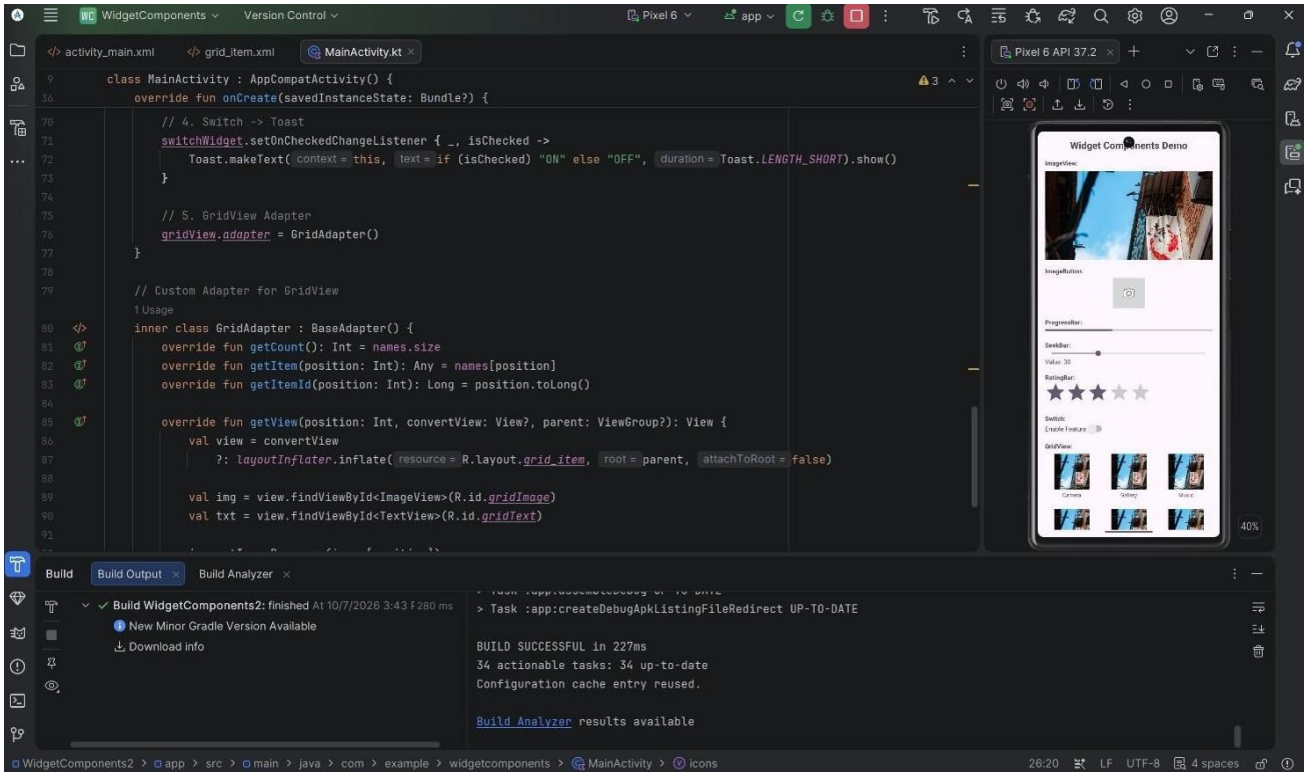
            img.setImageResource(icons[position])
            txt.text = names[position]
            return view
        }
    }
}
```

4.5 Building and Running the Application

The project was built using Gradle and compiled successfully, producing a BUILD SUCCESSFUL message.

The application was then installed and run on the Pixel 6 API 37.2 emulator.

Figure 5: Build Successful

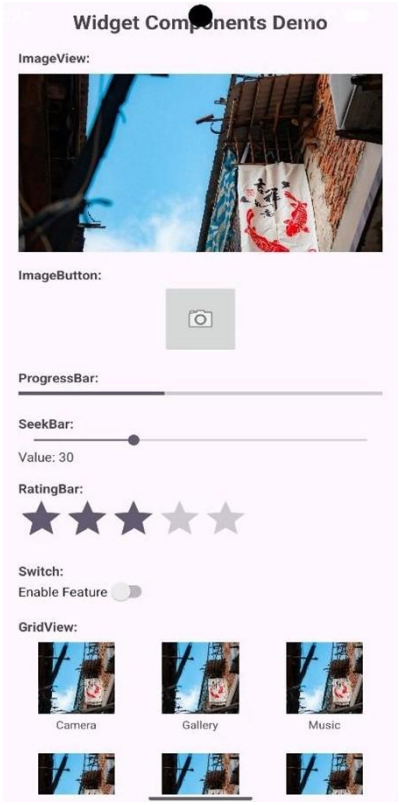


5. Results and Analysis

5.1 Application Output

After running the application on the Pixel 6 emulator, all widgets displayed correctly and functioned as designed.

Figure 6: Main Application Output



The output shows:

- **TextView (Title):** "Widget Components Demo"
- **ImageView:** displays a landscape image
- **ImageButton:** camera icon (clickable)
- **ProgressBar:** ~30% filled horizontal bar
- **SeekBar:** slider at Value = 30
- **RatingBar:** 3 out of 5 stars filled
- **Switch:** "Enable Feature" in OFF state
- **GridView:** 9 items in 3 columns

5.2 Test Cases and Results

Test Case	Action	Expected Result	Status
TC-01 – ImageButton	Tapped	ImageView-image changed	Pass
TC-02 – SeekBar	Dragged	ProgressBar & value updated	Pass
TC-03 – RatingBar	Tapped	Toast displayed rating	Pass
TC-04 – Switch	Toggled	Toast-displayed ON/OFF	Pass
TC-05 – ScrollView	Scrolled	All widgets visible	Pass
TC-06 – GridView	Viewed	All 9 items displayed	Pass

5.3 Analysis

From the results, the following observations were made:

- ScrollView enabled smooth scrolling when content exceeded screen height.
- Event handling worked correctly for all interactive widgets.
- The custom adapter (BaseAdapter) rendered GridView data efficiently by reusing convertView.
- findViewById() correctly linked XML views with Kotlin code.
- layoutInflater.inflate() properly rendered each grid cell.
- The build completed successfully with only 3 minor, non-functional warnings.

6. Summary

In this experiment, I successfully designed and implemented an Android application demonstrating the core UI widgets: ScrollView, ImageView, ImageButton, TextView, ProgressBar, SeekBar, RatingBar, Switch, and GridView. Through this experiment, I learned how to combine multiple widgets inside a ScrollView and LinearLayout, connect XML layouts with Kotlin using findViewById(), handle user events using listeners, and create a custom adapter by extending BaseAdapter. Overall, this experiment provided valuable hands-on experience in Android UI development and laid a strong foundation for building more advanced Android applications in the future.